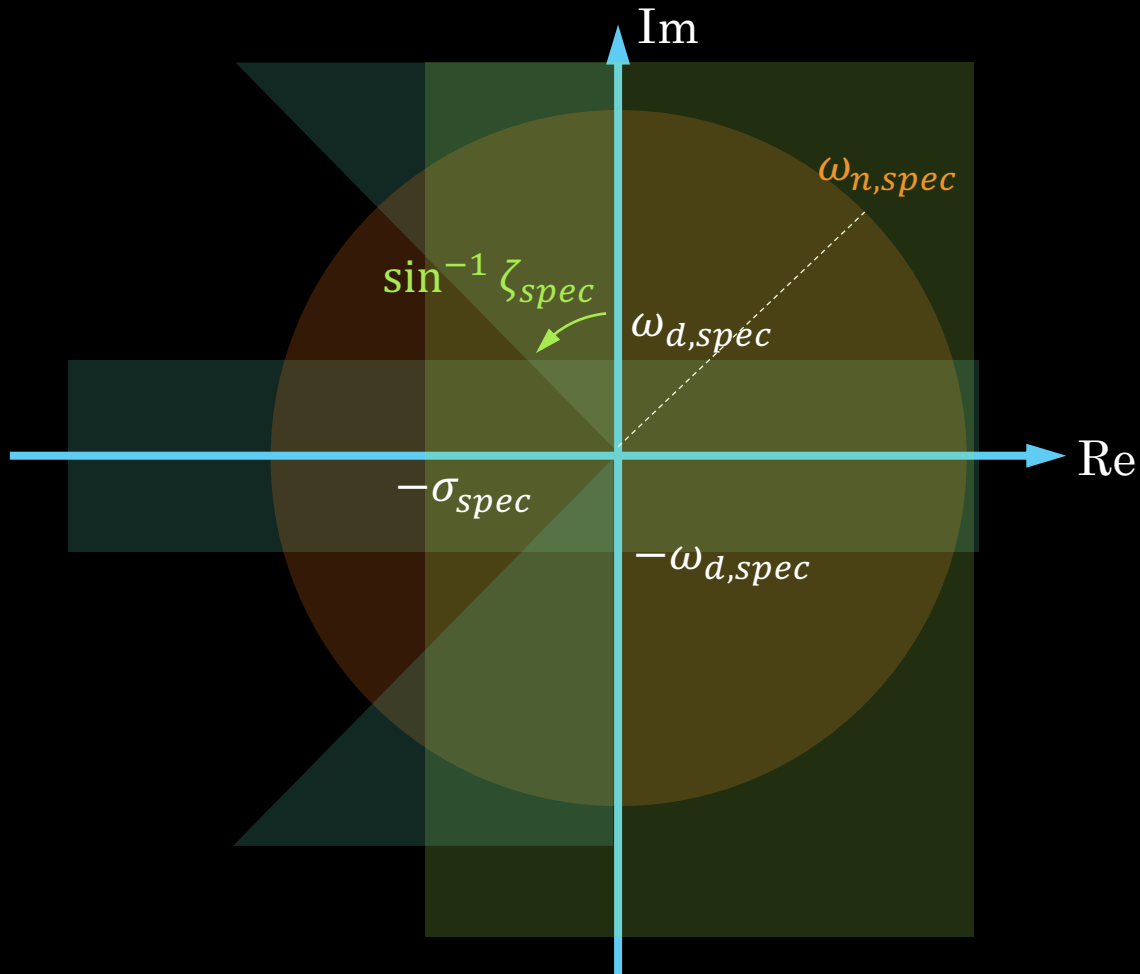


Optimization-based control

Control design given specifications



1% Settling time: $t_{1\%,s} = \frac{4.6}{\zeta \omega_n}$

Require: $t_s \leq t_{s,spec} \Rightarrow \sigma \geq \frac{4.6}{t_{s,spec}} = \sigma_{spec}$

Rise time: $t_r \approx \frac{1.8}{\omega_n}$

Require: $t_r \leq t_{r,spec} \Rightarrow \omega_n \geq \frac{1.8}{t_{r,spec}} = \omega_{n,spec}$

Peak time: $t_p = \frac{\pi}{\omega_n \sqrt{1-\zeta^2}}$

Require: $t_p \leq t_{p,spec} \Rightarrow \omega_d \geq \frac{\pi}{t_{p,spec}} = \omega_{d,spec}$

Maximum overshoot: $M_p = e^{\frac{-\pi\zeta}{\sqrt{1-\zeta^2}}}$

Require: $M_p \leq M_{p,spec} \Rightarrow \zeta^2 \geq \frac{(\ln M_{p,spec})^2}{\pi^2 + (\ln M_{p,spec})^2} = \zeta_{spec}^2$

Shaded areas mean not allowed

Specification- vs optimization-based control

Specification-based control

- Need controller to ensure a set of requirements are met
 - E.g., settling time, rise time, overshoot
- Unclear how to determine the “best” controller.

Optimization-based control

- Choose the *best* controller given a *cost function*
 - A is strictly better than B if $c(A) < c(B)$

Mathematical optimization

*The science of finding the **best** solution from a set of available alternatives*

minimize $c(x)$

Type equation here.

subject to $g_i(x) \leq 0, \quad i = 1, \dots, n_g$
 $h_j(x) = 0, \quad j = 1, \dots, n_h$
 $x \in \mathcal{X}$

Mathematical optimization

Supervised learning

$$\min_{\theta} \quad \frac{1}{N} \sum_{(x,y) \in \mathcal{D}} (y - m_{\theta}(x))^2$$

Mathematical optimization

Generative modeling

$$\min_{\theta} \quad \frac{1}{N} \sum_{x \in \mathcal{D}} -\log p_{\theta}(x)$$

Mathematical optimization

Resource allocation

$$\begin{array}{ll}\max_{x_1, x_2} & p_1 x_1 + p_2 x_2 \\ \text{subject to} & t_1 x_1 + t_2 x_2 \leq t_{\max} \\ & m_1 x_1 + m_2 x_2 \leq m_{\max} \\ & s_1 x_1 + s_2 x_2 \leq s_{\max}\end{array}$$

Types of optimization problems

Convex programs

- Linear program
- Quadratic program

Aside: Convex functions and convex sets

Types of optimization problems

Non-convex

- Mixed integer program
- Planning around obstacles

How to solve optimization problems

- Gradient descent
- Derivative-free methods
 - Sampling-based
- If convex, use beloved [cvxpy](#)
- Iterative methods: Make convex approximation, solve, then convexify about new solution

cvxpy

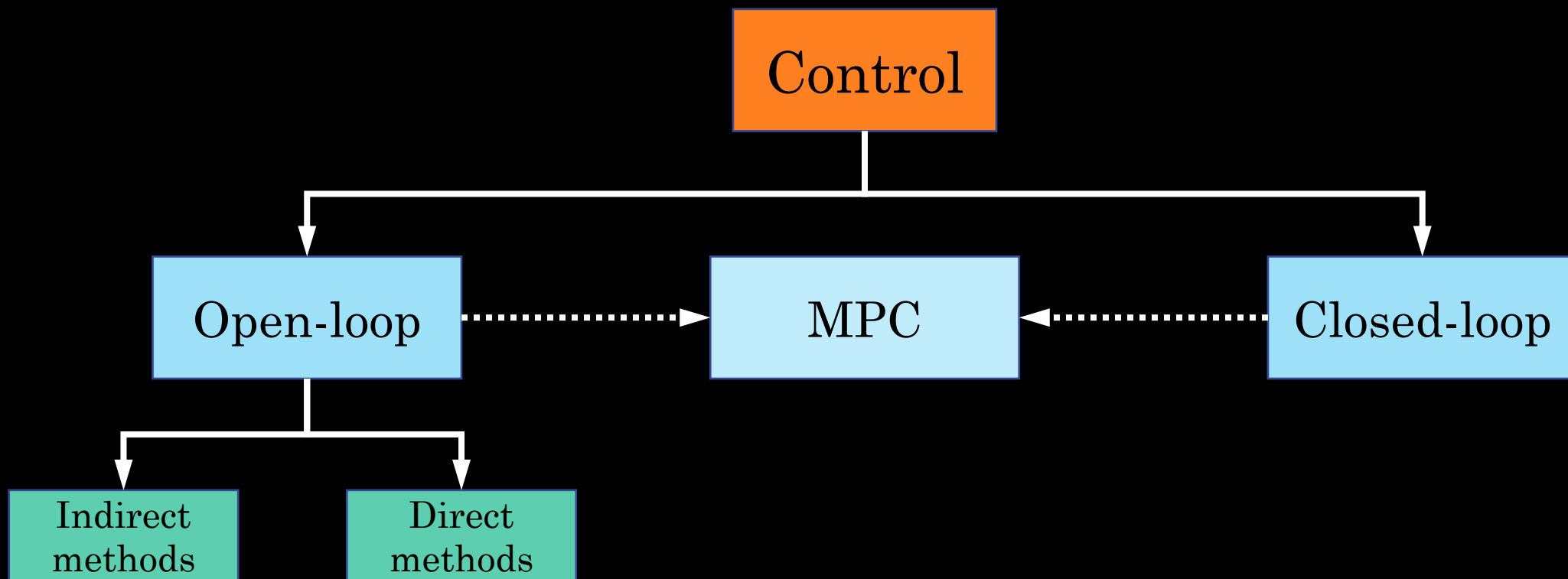
A domain-specific modeling language for (convex) mathematical optimization.

It lets you express the optimization problem in a natural way. User does not need to worry about the hairy details of the specific backend solver.

Trajectory optimization

Optimization-based control // Open-loop control

Additional reference: <https://underactuated.mit.edu/trajopt.html>



Optimal control problem: Control synthesis as an optimization problem

$$\begin{array}{ll} \min_{u(\cdot)} & \int_0^T c(x(\tau), u(\tau), \tau) d\tau + c_T(x(T), T) \\ \text{(CT) subject to} & \dot{x} = f(x, u, t) \\ & g_i(x(\tau), u(\tau)) \leq 0, \quad i = 1, \dots, n_g \\ & h_j(x(\tau), u(\tau)) = 0, \quad j = 1, \dots, n_h \\ & x(0) = x_{\text{current}} \\ & x \in \mathcal{X}, u \in \mathcal{U} \\ & \tau \in [0, T] \end{array}$$

Approaches to solving the continuous time optimal control problem

Indirect method (*“Optimize then discretize”*)

Use calculus of variations to solve for optimality conditions then solve for the conditions.

(Results in a two-point boundary value problem)

Direct method (*“Discretize then optimize”*)

Discretize the problem to obtain finite-dimensional optimization problem.

(Results in solving a mathematical optimization program)

Indirect method: Pontryagin's Minimum Principle (PMP)

Hamiltonian of the system

$$H(x, u, \lambda, t) = c(x, u, t) + \lambda^T f(x, u, t)$$

λ is the *costate*, which represents sensitivity of your cost to changes in the state ($\lambda = \nabla V(x, t)$)

First-order necessary conditions for optimality

$$\frac{\partial H}{\partial \lambda} = \dot{x}, \quad \frac{\partial H}{\partial x} = -\dot{\lambda}, \quad \frac{\partial H}{\partial u} = 0$$

Results in 2PBVP. Problem becomes more challenging with controls & state constraints.

Direct methods: Turn into finite-dimensional problem

Single Shooting

$$\begin{aligned} & \min_{u_0, \dots, u_{N-1}} \quad \sum_{k=0}^{N-1} c(x_k, u_k, t_k) \quad + \quad c_N(x_N, t_N) \\ & \text{subject to} \quad x_{k+1} = f(x_k, u_k, t_k) \quad k = 0, \dots, N-1 \\ & \quad \quad \quad g_i(x_k, u_k) \leq 0, \quad i = 1, \dots, n_g \\ & \quad \quad \quad h_j(x_k, u_k) = 0, \quad j = 1, \dots, n_h \\ & \quad \quad \quad x_0 = x_{\text{current}} \\ & \quad \quad \quad x_k \in \mathcal{X}, u_k \in \mathcal{U}, \quad k = 0, \dots, N \end{aligned}$$

Multiple shooting

$$\begin{aligned} & \min_{u_0, \dots, u_{N-1}, x_0, \dots, x_N} \sum_{k=0}^{N-1} c(x_k, u_k, t_k) + c_N(x_N, t_N) \\ \text{subject to } & x_{k+1} = f(x_k, u_k, t_k) & k = 0, \dots, N-1 \\ & g_i(x_k, u_k) \leq 0, & i = 1, \dots, n_g \\ & h_j(x_k, u_k) = 0, & j = 1, \dots, n_h \\ & x_0 = x_{\text{current}} \\ & x_k \in \mathcal{X}, u_k \in \mathcal{U}, & k = 0, \dots, N \end{aligned}$$

(Multiple) Shooting

Collocation method

Introduce **algebraic constraints** to enforce the system dynamics at those nodes.

e.g., Trapezoidal collocation

$$x_{k+1} = x_k + \frac{\Delta t}{2} (f(x_k, u_k) + f(x_{k+1}, u_{k+1}))$$

Similar thing for integral in objective

Pseudospectral

Represent control and states as *polynomials* and evaluate at quadrature nodes

$$x(t) = \sum_{j=0}^N c_j p_j(t), \quad u(t) = \sum_{j=0}^N d_j p_j(t)$$

- Use “special” polynomial basis and nodes for better numerical stability and properties
 - Chebyshev, Lagrange, Legendre

In all these methods, often results in *nonlinear program*

- The dynamics are nonlinear
- Objective is nonlinear
- Constraints are nonlinear

$$\begin{aligned} & \min_{u_0, \dots, u_{N-1}, x_0, \dots, x_N} \sum_{k=0}^{N-1} c(x_k, u_k, t_k) + c_N(x_N, t_N) \\ & \text{subject to} \quad x_{k+1} = f(x_k, u_k, t_k) \quad k = 0, \dots, N-1 \\ & \quad \quad \quad g_i(x_k, u_k) \leq 0, \quad i = 1, \dots, n_g \\ & \quad \quad \quad h_j(x_k, u_k) = 0, \quad j = 1, \dots, n_h \\ & \quad \quad \quad x_0 = x_{\text{current}} \\ & \quad \quad \quad x_k \in \mathcal{X}, u_k \in \mathcal{U}, \quad k = 0, \dots, N \end{aligned}$$

NLP solvers

- Use a nonlinear program solver
- Matlab `fmincon`

Description

Nonlinear programming solver.

Finds the minimum of a problem specified by

$$\min_x f(x) \text{ such that } \begin{cases} c(x) \leq 0 \\ ceq(x) = 0 \\ A \cdot x \leq b \\ Aeq \cdot x = beq \\ lb \leq x \leq ub, \end{cases}$$

NLP solvers

- Use a nonlinear program solver
- Matlab `fmincon`
- `scipy.optimize.minimize`

```
scipy.optimize.  
minimize
```

```
minimize(fun, x0, args=(), method=None, jac=None, hess=None, hessp=None,  
bounds=None, constraints=(), tol=None, callback=None, options=None)
```

Minimization of scalar function of one or more variables.

[\[source\]](#)

NLP solvers

- Use a nonlinear program solver
- Matlab `fmincon`
- `scipy.optimize.minimize`
- Many others, commercial solvers, free academic licenses.
- Lots of open-source ones too

But what is happening under the hood in these NLP solvers?

- Really smart with linear algebra and other tricks
 - E.g., exploiting sparsity, well-conditioning the problem
- Some sort of **log-barrier method**, turning constrained to unconstrained
- Apply some form of approximate method
- Smarter gradient descent
 - Use second order information, smarter line search

Sequential Quadratic Program (SQP)

$$\begin{aligned} \min_{u_0, \dots, u_{N-1}, x_0, \dots, x_N} \quad & \sum_{k=0}^{N-1} c(x_k, u_k, t_k) + c_N(x_N, t_N) \\ \text{subject to} \quad & x_{k+1} = f(x_k, u_k, t_k) \quad k = 0, \dots, N-1 \\ & g_i(x_k, u_k) \leq 0, \quad i = 1, \dots, n_g \\ & h_j(x_k, u_k) = 0, \quad j = 1, \dots, n_h \\ & x_0 = x_{\text{current}} \\ & x_k \in \mathcal{X}, u_k \in \mathcal{U}, \quad k = 0, \dots, N \end{aligned}$$

Sequential quadratic programming

- Approximate the problem as a quadratic program and repeatedly solve
 1. Start with initial guess $(\tilde{x}_0, \tilde{x}_1, \dots, \tilde{x}_{T+1}, \tilde{u}_0, \tilde{u}_1, \dots, \tilde{u}_T)$
 2. Linearize dynamics about guess $x_{t+1} \approx A_t x_t + B_t u_t + C_t$
 3. Linearize constraints $g(x, u) \leq 0$: $G_t x_t + H_t u_t + h_t \leq 0$
 4. Quadratize cost $c(x_t, u_t) \approx \frac{1}{2} x_t^T Q_t x_t + \frac{1}{2} u_t^T R_t u_t + p_t^T x_t + r_t^T u_t$
 5. Add trust region penalty to cost: $x_t \approx \tilde{x}_t, u_t \approx \tilde{u}_t$
 6. Solve QP! Solution = new guess
 7. Go to step 1.

Initialization matters



What if the problem is *really* hard?



Or when there's lots of dynamic obstacles



What are the challenge that arise?

- Cost, dynamics, constraints are *ugly* to work with
 - Difficult to differentiate
 - Maybe blackbox
 - Maybe learned (neural network)
- Stochastic elements
 - Uncertainty in the environment.
 - Require drawing many samples
- Generally, the standard NLP / gradient descent framework is not applicable.

Sampling-based: Model Predictive Path Integral (MPPI)

Information Theoretic MPC for Model-Based Reinforcement Learning

Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews,
James M. Rehg, Byron Boots, and Evangelos A. Theodorou

Abstract— We introduce an information theoretic model predictive control (MPC) algorithm capable of handling complex cost criteria and general nonlinear dynamics. The generality of the approach makes it possible to use multi-layer neural networks as dynamics models, which we incorporate into our MPC algorithm in order to solve model-based reinforcement learning tasks. We test the algorithm in simulation on a cart-pole swing up and quadrotor navigation task, as well as on actual hardware in an aggressive driving task. Empirical results demonstrate that the algorithm is capable of achieving a high level of performance and does so only utilizing data collected from the system.



Fig. 1. Aggressive driving with MPPI and neural network dynamics.

<https://www.overland.ai/>

<https://homes.cs.washington.edu/~bboots/files/InformationTheoreticMPC.pdf>

Sampling-based: Model Predictive Path Integral

1. Start with nominal trajectory $\tilde{\mathbf{u}}$
2. Add noise to it to generate many trajectories δ_k
3. Evaluate cost of each trajectory c_k
4. Compute weight for each trajectory $w_k = \frac{e^{-\frac{1}{\lambda}c_k}}{\sum_j e^{-\frac{1}{\lambda}c_j}}$
5. Compute weighted sum over controls to compute control $\mathbf{u}^* = \tilde{\mathbf{u}} + \sum_k w_k \delta_k$

Idea: Take a noisy estimate of the gradient and move along that direction

MPPI

Generating good warm starts for optimization



Flow matching

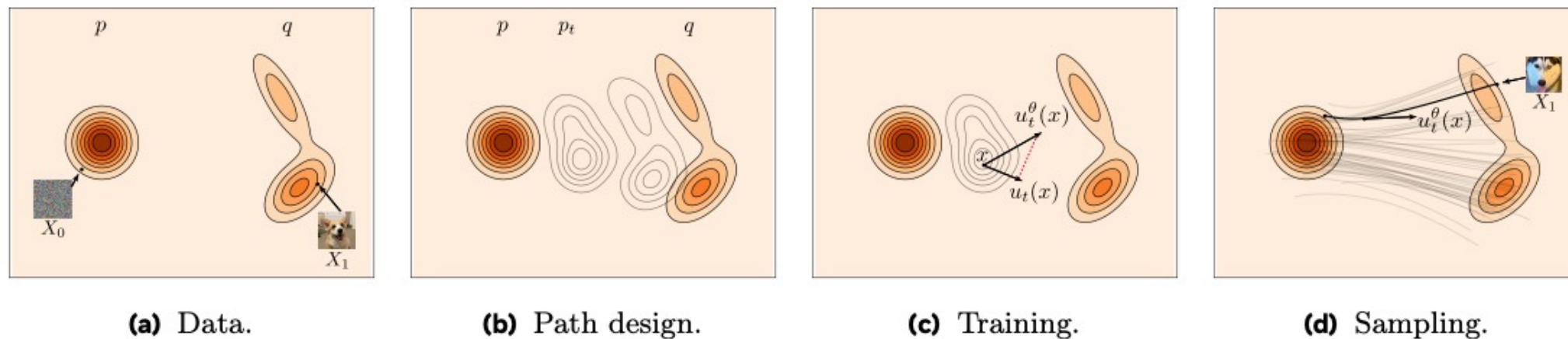


Figure 2 *The Flow Matching blueprint.* (a) The goal is to find a flow mapping samples X_0 from a known source or noise distribution p into samples X_1 from an unknown target or data distribution q . (b) To do so, design a time-continuous probability path $(p_t)_{0 \leq t \leq 1}$ interpolating between $p := p_0$ and $q := p_1$. (c) During training, use regression to estimate the velocity field u_t known to generate p_t . (d) To draw a novel target sample $X_1 \sim q$, integrate the estimated velocity field $u_t^\theta(X_t)$ from $t = 0$ to $t = 1$, where $X_0 \sim p$ is a novel source sample.

$$\mathcal{L}_{\text{CFM}}^{\text{OT, Gauss}}(\theta) = \mathbb{E}_{t, X_0, X_1} \|u_t^\theta(X_t) - (X_1 - X_0)\|^2, \text{ where } t \sim U[0, 1], X_0 \sim \mathcal{N}(0, I), X_1 \sim q.$$

<https://ai.meta.com/research/publications/flow-matching-guide-and-code/>

<https://dl.heeere.com/conditional-flow-matching/blog/conditional-flow-matching/>